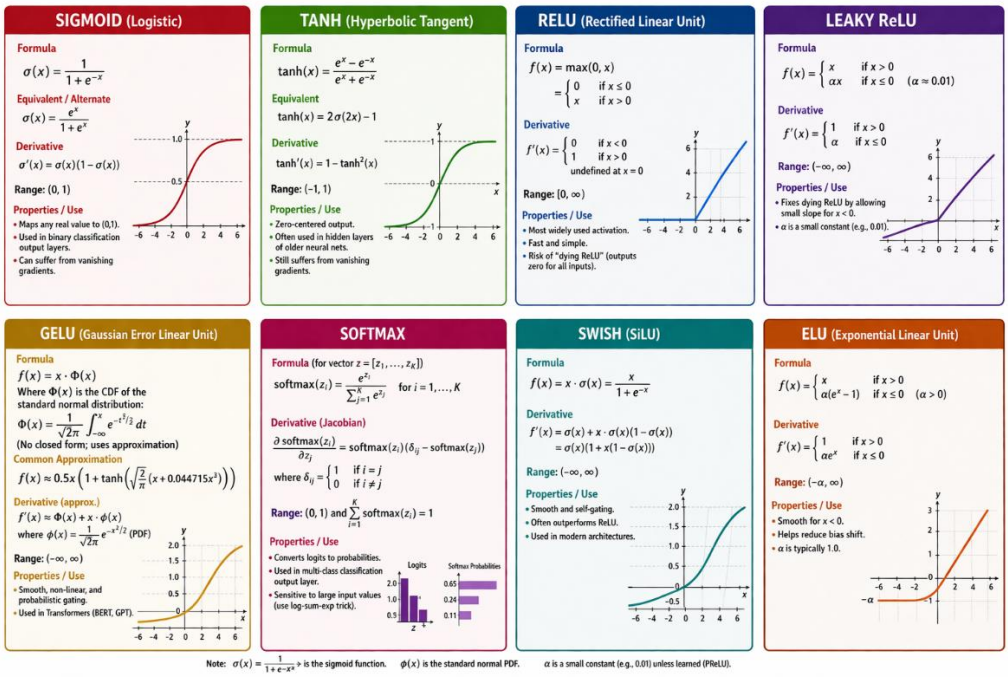
ACTIVATION FUNCTIONS



LOSS FUNCTIONS

- Binary Cross Entropy loss function

$$-\left[y_t \cdot \log y_p + (1-y_t) \cdot \log(1-y_p)\right]$$

- Cross Entropy loss function

$$-\sum_{i=1}^{n} y_{t_i} \cdot \log y_{p_i}$$

- Weighted Cross Entropy loss function

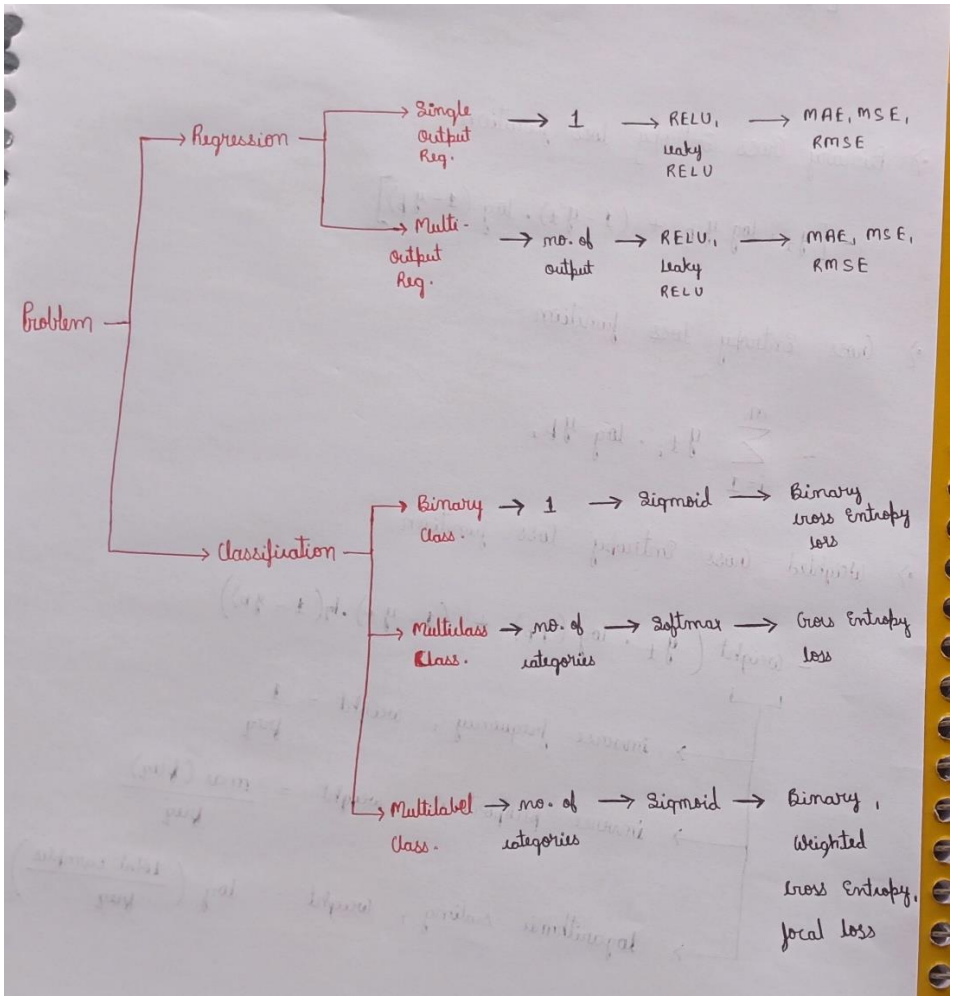
$$-\text{weight}\left(y_t \cdot \log(y_p) + (1-y_t) \cdot \log(1-y_p)\right)$$

- → inverse frequency, weight $= \frac{1}{\text{freq}}$
- → inverse proportion, weight $= \frac{\max(\text{freq})}{\text{freq}}$
- → logarithmic scaling, weight $= \log\left(\frac{\text{total samples}}{\text{freq}}\right)$

- Focal loss

$$-\alpha\left(y_t \cdot (1-y_p)^{\gamma} + (1-y_t) \cdot y_p^{\gamma}\right)$$

TUNING OF OUTPUT LAYER



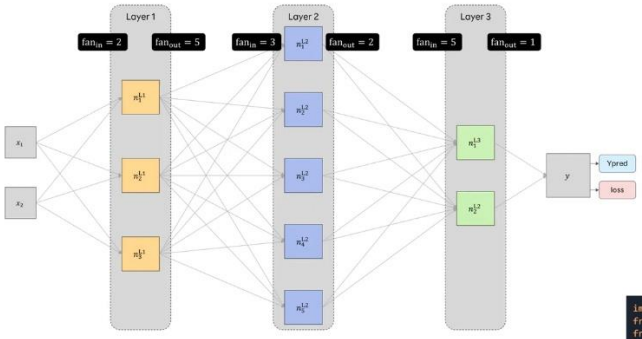
TUNING OF HIDDEN LAYER

Reasons for Proper Weight Initialization

1. **Prevents Vanishing Gradients** → Ensures gradients don't shrink to zero in deep networks.
2. **Prevents Exploding Gradients** → Keeps gradients from becoming excessively large and unstable.
3. **Ensures Faster Convergence** → Helps the model train quicker by starting in a good region of the loss surface.
4. **Breaks Symmetry** → Allows different neurons to learn different features instead of identical outputs.
5. **Maintains Stable Activation Variance** → Keeps activations consistent across layers, enabling smooth learning.

Methods to initialize the weights properly are –

Weights – Xavier/Glorot Initialization



Raw Pytho

```python
import numpy as np
import math
import random

def xavier_normal(fan_in=1, fan_out=1):
    variance = 2.0 / (fan_in + fan_out)
    stddev = np.sqrt(variance)
    weights = np.random.normal(0, stddev, size=(fan_in))
    return weights

def xavier_uniform(fan_in, fan_out):
    high = math.sqrt(6.0 / (fan_in + fan_out))
    low = -high
    weights = np.array([random.uniform(low, high) for _ in range(fan_in)])
    return weights

print("Xavier Normal Distribution:")
xavier_normal(fan_in=2, fan_out=5)  # Layer 1
xavier_normal(fan_in=3, fan_out=2)  # Layer 2
xavier_normal(fan_in=5, fan_out=1)  # Layer 3

print("Xavier Uniform Distribution:")
xavier_uniform(fan_in=2, fan_out=5)  # Layer 1
xavier_uniform(fan_in=3, fan_out=2)  # Layer 2
xavier_uniform(fan_in=5, fan_out=1)  # Layer 3
```

Using Keras Library

```python
import numpy as np
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense

x = np.array([[i for i in range(1000)], [i for i in range(1000)]])
y = np.array([2*i + 3 for i in x])

model = Sequential()
model.add(Dense(3, activation='sigmoid', kernel_initializer='glorot_normal'))
model.add(Dense(5, activation='tanh', kernel_initializer="glorot_normal"))
model.add(Dense(2, activation='tanh', kernel_initializer="glorot_uniform"))
model.add(Dense(1, activation='linear', kernel_initializer="glorot_uniform"))

model.compile(loss='mse')

model.fit(x, y, epochs=50, batch_size=10)
```

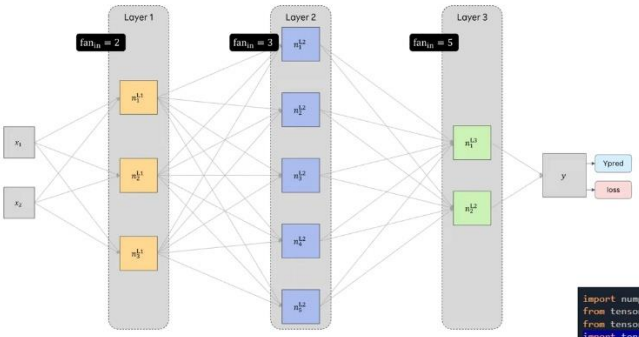
Xavier Normal

$$\left[\boldsymbol{mean} = 0, \boldsymbol{stdev} = \sqrt{\frac{2}{(\text{fan}_{in} + \text{fan}_{out})}}\right]$$

Xavier Uniform

$$\left[\boldsymbol{min} = \frac{-\sqrt{6}}{\sqrt{(\text{fan}_{in} + \text{fan}_{out})}}, \boldsymbol{max} = \frac{\sqrt{6}}{\sqrt{(\text{fan}_{in} + \text{fan}_{out})}}\right]$$

Weights – He Initialization



Raw Python

```python
def he_normal(fan_in):
    variance = 2 / fan_in
    stddev = np.sqrt(variance)
    weights = np.random.normal(0, stddev
    return weights

def he_uniform(fan_in):
    high = np.sqrt(6 / fan_in)
    low = -high
    weights = np.random.uniform(low, high, size=fan_in)
    return weights

print("He Normal Distribution:")
xavier_normal(fan_in=2, fan_out=5)  # Layer 1
xavier_normal(fan_in=3, fan_out=2)  # Layer 2
xavier_normal(fan_in=5, fan_out=1)  # Layer 3

print("He Uniform Distribution:")
xavier_uniform(fan_in=2, fan_out=5)  # Layer 1
xavier_uniform(fan_in=3, fan_out=2)  # Layer 2
xavier_uniform(fan_in=5, fan_out=1)  # Layer 3
```

Using Keras Library

```python
import numpy as np
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
import tensorflow.keras.initializers as Initializers

x = np.array([[i for i in range(1000)], [i for i in range(1000)]])
y = np.array([2*i + 3 for i in x])

model = Sequential()
model.add(Dense(3, activation='relu', kernel_initializer='he_normal'))
model.add(Dense(5, activation='relu', kernel_initializer="he_uniform"))
model.add(Dense(2, activation='relu', kernel_initializer=Initializers.HeNormal()))
model.add(Dense(1, activation='linear', kernel_initializer=Initializers.HeUniform()))

model.compile(loss='mse')

model.fit(x, y, epochs=50, batch_size=10)
```

He Normal

$$\left[\boldsymbol{mean} = 0, \boldsymbol{stdev} = \sqrt{\frac{2}{\text{fan}_{in}}}\right]$$

He Uniform

$$\left[\boldsymbol{min} = \frac{-\sqrt{6}}{\sqrt{\text{fan}_{in}}}, \boldsymbol{max} = \frac{\sqrt{6}}{\sqrt{\text{fan}_{in}}}\right]$$

Choosing between Xavier and He Initialization

- **When to choose Xavier**
  - If activation function is `sigmoid()` use Xavier Uniform
  - If activation function is `tanh()` use Xavier Normal
- **When to choose He**
  - If activation function are in the `relu()` class use He Normal
  - If He normal doesn't work or is sub-optimal, then use He Uniform

Bias is always initialized from the value of Zero .